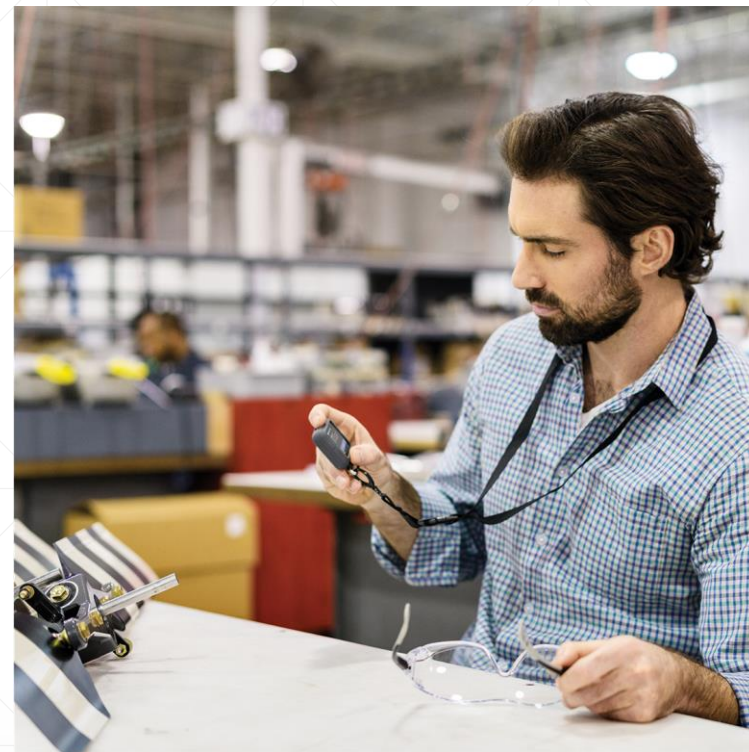


“锁”之二



显式锁与ReentrantLock

北京理工大学计算机学院
金旭亮

Lock接口



I Lock

(m) Lock	void
(m) lockInterruptibly()	void
(m) newCondition()	Condition
(m) tryLock()	boolean
(m) tryLock(long, TimeUnit)	boolean
(m) unlock()	void

实现了Lock接口的锁，称为“**显式锁**”。

Lock接口定义了比synchronized更强大的功能，开发中常用的实现了这一接口的类，有“ReentrantLock（可重入锁）”和“ReadWriteLock（读写锁）”两个。

可重入锁—ReentrantLock

ReentrantLock		
m	getHoldCount()	int
m	getQueueLength()	int
m	getWaitQueueLength(Condition)	int
m	hasQueuedThread(Thread)	boolean
m	hasQueuedThreads()	boolean
m	hasWaiters(Condition)	boolean
m	isFair()	boolean
m	isHeldByCurrentThread()	boolean
m	isLocked()	boolean
m	lock()	void
m	lockInterruptibly()	void
m	newCondition()	Condition
m	toString()	String
m	tryLock()	boolean
m	tryLock(long, TimeUnit)	boolean
m	unlock()	void

- 实例化ReentrantLock锁

```
Lock myLock = new ReentrantLock();
```

- 使用ReentrantLock锁可以保证一次只有一个线程执行临界区中的代码：

```
myLock.lock();
try{
    临界区中的代码
}
finally {
    myLock.unlock();
}
```

- 一个线程可以多次申请ReentrantLock锁（但必须保证它必须释放同样多次）。所以称它为“**可重入**”的锁。

掌握ReentrantLock的用法

```
public class WhatIsLock {  
    //Lock是一个接口  
    private final Lock lock = new ReentrantLock();  
    //将被多线程执行的线程函数  
    public void threadFunc() {  
        lock.lock();  
        for (int i = 0; i < 5; i++) {  
            System.out.println(Thread.currentThread().getName() + "处理 " + (i + 1));  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
        System.out.println("=====");  
        lock.unlock();  
    }  
}
```

```
public static void main(String[] args) {  
    var obj = new WhatIsLock();  
    for (int i = 1; i <= 4; i++) {  
        //谁抢到锁, 谁输出。  
        new Thread(obj::threadFunc).start();  
    }  
}
```

ReentrantLock中定义的方法isLocked()可用于检测相应锁是否被当前线程持有, getQueueLength()方法可用于获取等待该锁的线程的数量。

WhatIsLock.java

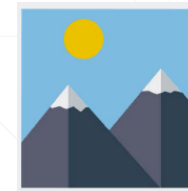
示例输出



```
Run: WhatIsLock x
"C:\Program Files\Java\
Thread-1处理 1
Thread-1处理 2
Thread-1处理 3
Thread-1处理 4
Thread-1处理 5
=====
Thread-0处理 1
Thread-0处理 2
Thread-0处理 3
Thread-0处理 4
Thread-0处理 5
=====
```

几个线程申请同一把锁，谁手快谁就先执行，执行完毕之后，它释放锁，才轮到其他的线程执行。

小结



内部锁

内部锁是基于代码块的锁，其使用基本无灵活性可言：要么用，要么不用，别无他选。

简单易用，且不会导致锁泄漏。

内部锁仅支持非公平锁

显式锁

显式锁是基于对象的锁，其使用可以充分发挥面向对象编程的灵活性

使用显式锁的时候必须注意将锁的释放操作放在`finally`块中，这点容易被开发者忽略。

显式锁既支持非公平锁，又支持公平锁。

显式锁支持了一些内部锁所不支持的特性。